

Comprehensive Security Setup for ShelfMaster (Node.js/Express/Supabase)

Line Spacing: 1.25

This document provides a step-by-step guide to implementing the enhanced security architecture for the ShelfMaster library system. These steps ensure that the backend protects sensitive data while maintaining functionality for both students and librarians.

Step 1: Environment Variables Configuration

The security of the application relies heavily on secret keys. These must never be hardcoded in the script. In Replit, use the **Secrets (Environment Variables)** tool to set the following:

Variable Name	Purpose	Action
SUPABASE_URL	Your Supabase project URL	Retrieve from Supabase Dashboard -> Settings -> API.
SUPABASE_SERVICE_ROLE_KEY	The private key that bypasses RLS	Strictly Confidential. Do not share or expose to the frontend.
JWT_SECRET	Key used to sign and verify user tokens	Generate a long, random string (e.g., using a password generator).

Step 2: Update the Database Query Endpoint

The core vulnerability in the original system was a "Transparent Proxy" flaw where any user could modify any table. The updated /api/db/query endpoint implements Role-Based Access Control (RBAC) and data sandboxing.

Implementation Logic:

- **Identity Verification:** Every request is parsed for a JWT. If no valid user is found, the request is rejected with a 401 Unauthorized status.
- **Role Check:** The server queries the users table to check the requester's role. If a non-librarian attempts a write action (insert, update, delete), the server returns a 403 Forbidden status.
- **Data Sandboxing (The "Jailer"):** If a student queries sensitive tables like transactions or fines, the server automatically injects a user_id filter into the query.

Step 3: Implement Rate Limiting

To prevent brute-force attacks on login and signup routes, install and configure `express-rate-limit`.

```
// Install: npm install express-rate-limit

import rateLimit from 'express-rate-limit';

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 15, // Limit each IP to 15 attempts
  message: { error: "Too many attempts. Please try again later." }
});

app.use('/api/auth/login', authLimiter);
app.use('/api/auth/signup', authLimiter);
```

Step 4: Database-Level Security (Supabase RLS)

Even with a secure API, you should enable Row Level Security (RLS) in the Supabase Dashboard as a second line of defense. This prevents unauthorized access even if someone discovers your `SUPABASE_URL`.

1. Navigate to the **Authentication** section and ensure `auth_users` is linked to your users table via `auth_id`.
2. Enable RLS on all tables (books, transactions, users, etc.).
3. Create policies that allow librarians full access and students "Read Only" or "Own Data Only" access.

Step 5: Deployment and Maintenance

Once the code is updated in your `server.js` file, follow these final checks:

- **Verify Column Names:** Ensure the column linking transactions to users is exactly `user_id`.
- **Audit Dependencies:** Periodically run `npm audit` to check for vulnerabilities in third-party libraries.
- **Rotate Secrets:** If you suspect a breach (e.g., local malware infection), immediately change your `JWT_SECRET` and `SUPABASE_SERVICE_ROLE_KEY` in Replit.